

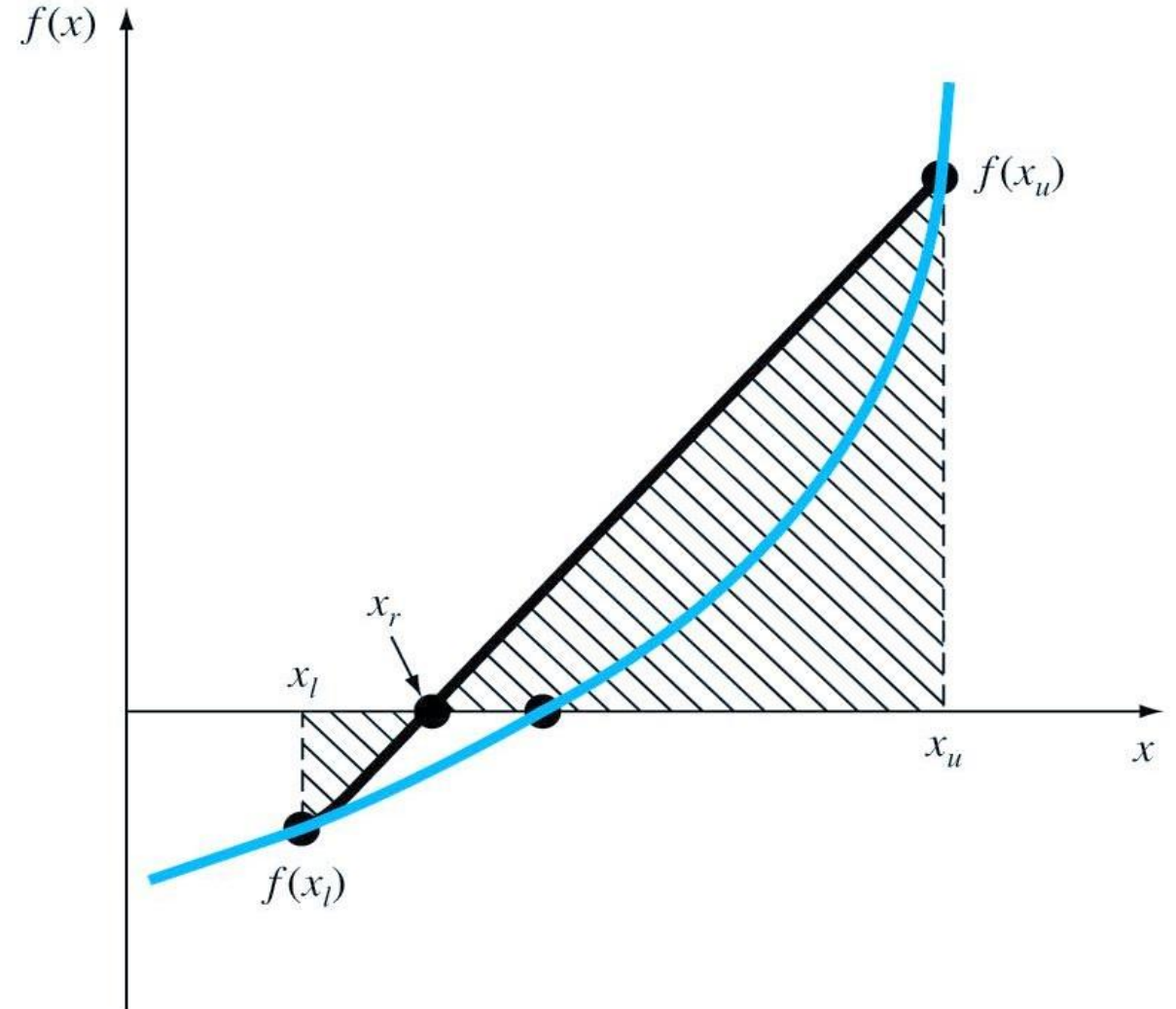
Lecture Slide - 4

False Position Method

The False-Position Method (Regula-Falsi)

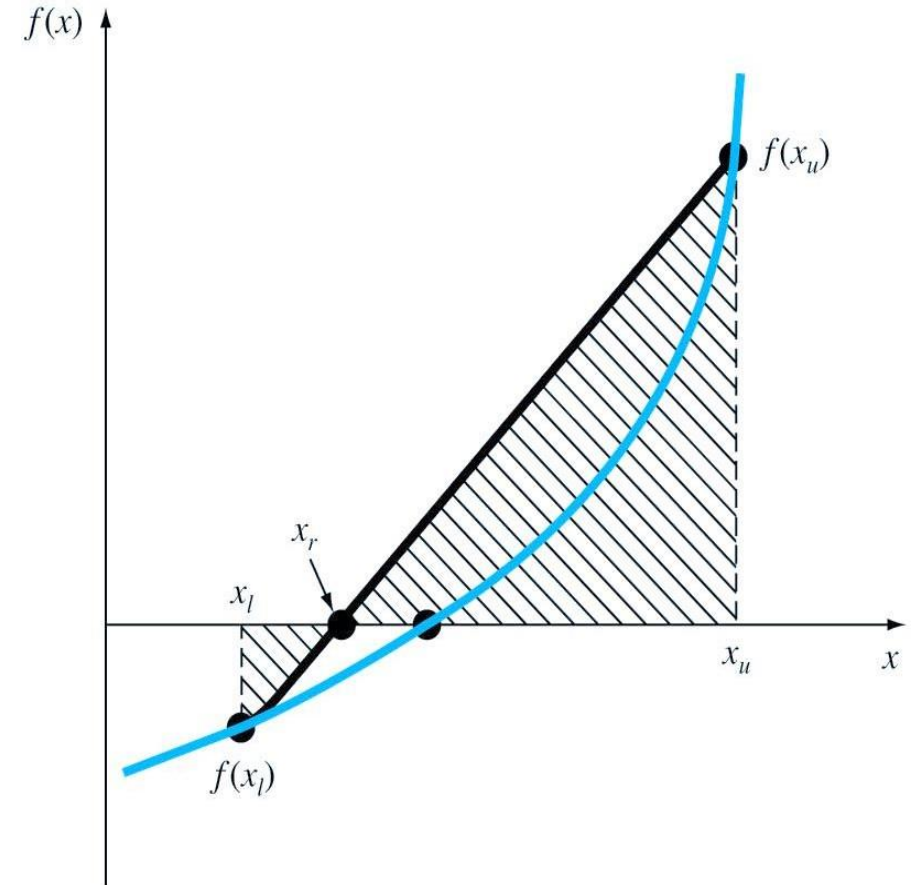
- If a real root is bounded by x_l and x_u of $f(x)=0$, then we can approximate the solution by doing a linear interpolation between the points $[x_l, f(x_l)]$ and $[x_u, f(x_u)]$ to find the x_r value such that $g(x_r)=0$, $g(x)$ is the linear approximation of $f(x)$.

== > Fig



The False-Position Method (Regula-Falsi)

- In this figure, the bisection method does not take into consideration that $f(x_L)$ is much closer to zero compared to $f(x_U)$
- In other words, the next predicted root x_r would be closer to x_L than the midpoint of x_L and x_U
- The false-position method takes advantage of this observation mathematically by drawing a secant from the function value at x_L to the function value at x_U and estimates the root where it crosses the x-axis.



== > Fig

Procedure

1. Find a pair of values of x , x_l and x_u such that

$$f_l = f(x_l) < 0 \text{ and } f_u = f(x_u) > 0.$$

2. Estimate the value of the root from the following formula

$$\frac{f(x_u) - f(x_r)}{x_u - x_r} = \frac{f(x_u) - f(x_l)}{x_u - x_l}$$

$$\text{since } f(x_r) = 0, \quad \frac{f_u}{x_u - x_r} = \frac{f_u - f_l}{x_u - x_l} \quad \Rightarrow \quad x_r = x_u - \frac{f_u(x_u - x_l)}{f_u - f_l}$$

$$\text{finally,} \quad x_r = \frac{x_l f_u - x_u f_l}{f_u - f_l}$$

Procedure

3. Use the new point to replace one of the original points, keeping the two points on opposite sides of the x-axis.

a. If $f(x_l) \times f(x_r) < 0$ (root is between x_l and x_r):

- The root is in the lower interval
- Set $x_u = x_r$ (replace the upper bound)

b. If $f(x_l) \times f(x_r) > 0$ (root is between x_r and x_u):

- The root is in the upper interval
- Set $x_l = x_r$ (replace the lower bound)

c. If $f(x_r) = 0$, we've found the exact root.

4. See if the new x_l and x_u are close enough for convergence to be declared. If they are not go back to step 2.

Why this method?

- Faster
- Always converges for a single root.
- Although false position often performs better than bisection, there are other cases where it does not.
- As in the following example $f(x) = x^{10} - 1$, there are certain cases where bisection yields superior results.

Advantages

1. It does not require the derivative calculation.
2. This method has first order rate of convergence i.e. it is linearly convergent. It always converges.

Disadvantages

Following are the limitations of regula falsi method

1. As it is trial and error method in some cases it may take large time span to calculate the correct root and thereby slowing down the process.
2. It is used to calculate only a single unknown in the equation.

Problem Solving

Use the **False-Position Method** to determine the drag coefficient c_d needed so that an 80kg bungee jumper has a velocity of 36 m/s after 4 seconds of free fall.

Note: The acceleration of gravity is 9.81 m/s^2 . Start with initial guesses of (c_d) $x_l = 0.1$ and $x_u = 0.2$ and iterate until the approximate relative error falls below 2%.

Solution: The velocity of a falling bungee jumper is

$$v(c_d) = \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}} t\right) \text{ or, } f(c_d) = \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}} t\right) - v(c_d)$$

$$\text{or, } f(c_d) = \sqrt{\frac{9.81 \times 80}{c_d}} \tanh\left(\sqrt{\frac{9.81 \times c_d}{80}} 4\right) - 36 \quad \dots(1)$$

Problem Solving

Given, $c_d(\text{lower}) = x_l = 0.1$ and $c_d(\text{upper}) = x_u = 0.2$

Iteration – 1, $x_l = 0.1$ and $x_u = 0.2$

From equation 1, $f_l = f(0.1) = 0.86029$ and $f_u = f(0.2) = -1.19738$

$$x_r = \frac{0.1 \times (-1.19738) - 0.2 \times 0.8602906}{-1.19738 - 0.8602906} = 0.141808957$$

$f(x_r) = -0.03521093$, **(-ve)** therefore new $x_u = x_r = 0.141808957$

Iteration – 2, $x_l = 0.1$ and $x_u = 0.141808957$

$f_l = f(0.1) = 0.86029$ and $f_u = f(0.141808957) = -0.03521093$

$$x_r = \frac{0.1 \times (-0.03521093) - 0.141808957 \times 0.8602906}{-0.03521093 - 0.8602906} = 0.140165037$$

[Old approximation = old $x_r = x_u$] error = $((x_r - x_u)/x_r) * 100\% = 1.17\%$

Iteration		$f(c_d)$	x_r	error
1	x_u	-1.197380001		
2	x_l	0.860290613	0.141808957	1.17%
3	x_u	-0.035210927	0.140165037	
4	x_u	-0.000996447	0.140118569	
5	x_u	-2.81676E-05	0.140117256	
6	x_u	-7.96218E-07	0.140117219	

More Example

- Find the root of $f(x) = (x-4)^2(x+2) = 0$ using the initial guesses of $x_L = -2.5$ and $x_U = -1.0$ and a pre-specified tolerance of $\epsilon_s = 0.1\%$

- Check initial guesses

- $x_L = -2.5, x_U = -1.0$
- $f(x_L) = (-2.5 - 4)^2(-2.5 + 2) = (-6.5)^2 \times (-0.5) = 42.25 \times (-0.5) = -21.125$
- $f(x_U) = (-1.0 - 4)^2(-1.0 + 2) = (-5)^2 \times 1 = 25 \times 1 = 25$

Since $f(x_L) < 0$ and $f(x_U) > 0$, the root lies in $[-2.5, -1.0]$.

- The approximate relative error:

$$\epsilon_a = \left| \frac{x_r^{\text{new}} - x_r^{\text{old}}}{x_r^{\text{new}}} \right| \times 100\%$$

More Example

- Find the root of $f(x) = (x-4)^2(x+2) = 0$ using the initial guesses of $x_L = -2.5$ and $x_U = -1.0$ and a pre-specified tolerance of $\epsilon_s = 0.1\%$

Iteration 1:

- $x_L = -2.5, f_L = -21.125$
- $x_U = -1.0, f_U = 25$

$$x_r = \frac{(-2.5)(25) - (-1.0)(-21.125)}{25 - (-21.125)} = \frac{-62.5 - 21.125}{46.125} = \frac{-83.625}{46.125} = -1.813$$

Now, $f(x_r) = f(-1.813) = (-1.813 - 4)^2(-1.813 + 2) = (-5.813)^2 \times (0.187) = 33.79 \times 0.187 = 6.318$

Since $f(x_r) > 0$, we update $x_U = x_r = -1.813$ and $f_U = 6.318$.

Now, $x_L = -2.5, x_U = -1.813$

No error calculation for the first iteration.

More Example

- Find the root of $f(x) = (x-4)^2(x+2) = 0$ using the initial guesses of $x_L = -2.5$ and $x_U = -1.0$ and a pre-specified tolerance of $\epsilon_s = 0.1\%$

Iteration 2:

- $x_L = -2.5, f_L = -21.125$
- $x_U = -1.813, f_U = 6.318$

$$x_r = \frac{(-2.5)(6.318) - (-1.813)(-21.125)}{6.318 - (-21.125)} = \frac{-15.795 - 38.279}{27.443} = \frac{-54.074}{27.443} = -1.970$$

$$f(x_r) = f(-1.970) = (-1.970 - 4)^2(-1.970 + 2) = (-5.97)^2 \times (0.03) = 35.64 \times 0.03 = 1.069$$

Since $f(x_r) > 0$, update $x_U = x_r = -1.970, f_U = 1.069$

Now, $x_L = -2.5, x_U = -1.970$

Calculate error:

$$\epsilon_a = \left| \frac{-1.970 - (-1.813)}{-1.970} \right| \times 100\% = \left| \frac{-0.157}{-1.970} \right| \times 100\% = 7.97\%$$

More Example

- Find the root of $f(x) = (x-4)^2(x+2) = 0$ using the initial guesses of $x_L = -2.5$ and $x_U = -1.0$ and a pre-specified tolerance of $\epsilon_s = 0.1\%$

Iteration 3:

- $x_L = -2.5, f_L = -21.125$
- $x_U = -1.970, f_U = 1.069$

$$x_r = \frac{(-2.5)(1.069) - (-1.970)(-21.125)}{1.069 - (-21.125)} = \frac{-2.6725 - 41.586}{22.194} = \frac{-44.2585}{22.194} = -1.994$$

$$f(x_r) = f(-1.994) = (-1.994 - 4)^2(-1.994 + 2) = (-5.994)^2 \times (0.006) = 35.928 \times 0.006 = 0.2156$$

Since $f(x_r) > 0$, update $x_U = x_r = -1.994, f_U = 0.2156$

Now, $x_L = -2.5, x_U = -1.994$

Error:

$$\epsilon_a = \left| \frac{-1.994 - (-1.970)}{-1.994} \right| \times 100\% = \left| \frac{-0.024}{-1.994} \right| \times 100\% = 1.20\%$$

More Example

- Find the root of $f(x) = (x - 4)^2(x + 2) = 0$ using the initial guesses of $x_L = -2.5$ and $x_U = -1.0$ and a pre-specified tolerance of $\epsilon_s = 0.1\%$

Table 2: Root of $f(x) = (x - 4)^2(x + 2) = 0$ the false-position method

Iteration	x_L	x_U	$f(x_L)$	$f(x_U)$	x_r	$ \epsilon_a \%$	$f(x_r)$
1	-2.5	-1	-21.13	25.00	-1.813	N/A	6.319
2	-2.5	-1.813	-21.13	6.319	-1.971	7.97	1.069
3	-2.5	-1.971	-21.13	1.028	-1.994	1.20	0.2156
4	-2.5	-1.996	-21.13	0.1542	-1.999	0.25	0.03599
5	-2.5	-1.999	-21.13	0.02286	-2.000	0.05	0.0

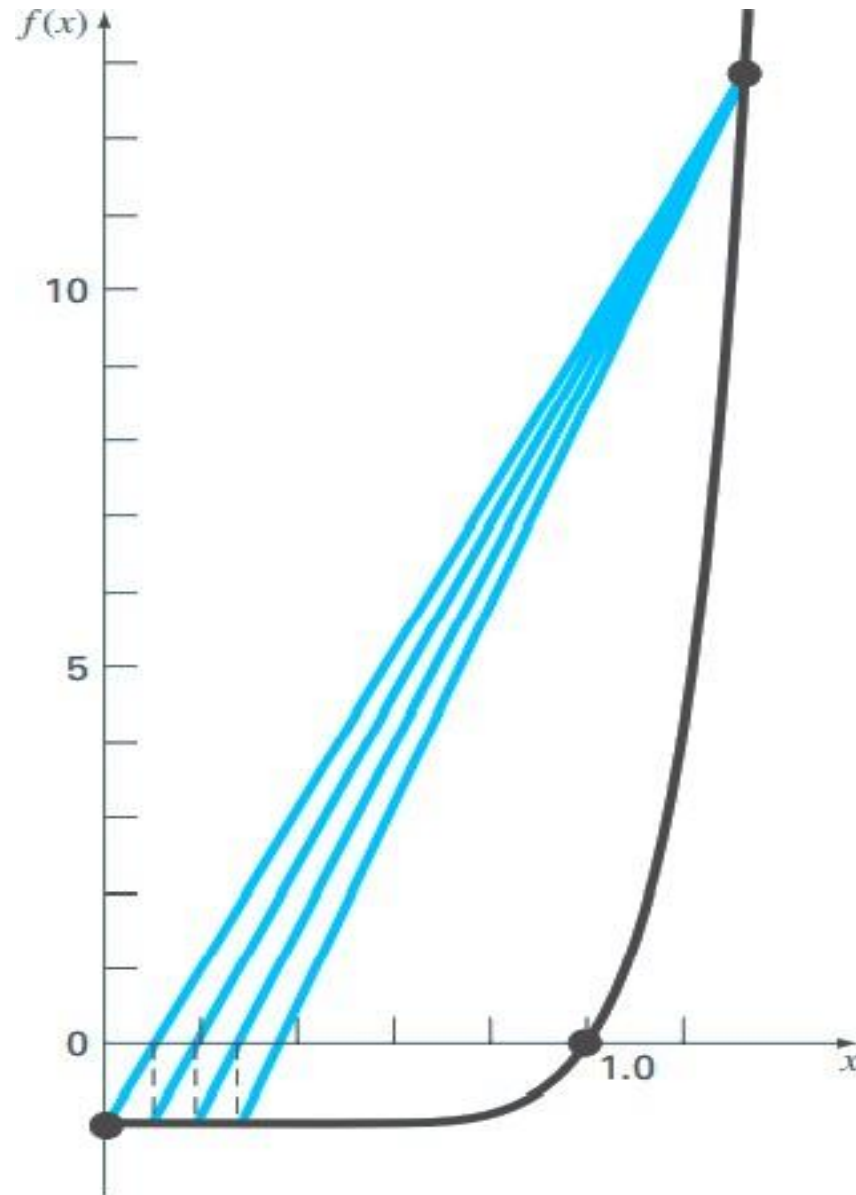
Comparison Bisection vs False Position

$$f(x) = 3x + \sin x - e^x = 0$$

$$x_0 = 0, x_1 = 1, f_0 \times f_1 < 0$$

k	x_k (Bisection)	f_k		x_k (False Position)	f_k
1	0.5			0.471	
2	0.25			0.372	
3	0.375			0.362	
4	0.3125			0.360	
5	0.34315	-0.042		0.360	2.93×10^{-5}

Certain Case



Plot of $f(x) = x^{10} - 1$, shows slow convergence of the false-position method

Bisection Method (Converge quicker)

Iteration	x_l	x_u	x_r	ε_a (%)	ε_t (%)
1	0	1.3	0.65		35
2	0.65	1.3	0.975	33.3	25
3	0.975	1.3	1.1375	14.3	13.8
4	0.975	1.1375	1.05625	7.7	5.6
5	0.975	1.05625	1.015625	4.0	1.6

False-position Method

Iteration	x_l	x_u	x_r	ε_a (%)	ε_t (%)
1	0	1.3	0.09430		90.6
2	0.09430	1.3	0.18176	48.1	81.8
3	0.18176	1.3	0.26287	30.9	73.7
4	0.26287	1.3	0.33811	22.3	66.2
5	0.33811	1.3	0.40788	17.1	59.2

Why Bisection Is Better here?

In this case:

- At $a = 0$, $f(0) = -1$.
- At $b = 2$, $f(2) = 2^{10} - 1 = 1023$.
- The values are **highly unbalanced** (-1 vs 1023).
- The false position formula gives c very close to $a = 0$.
- As iterations proceed, the left endpoint a barely moves, and convergence becomes extremely **slow and biased toward one side**.

Why Bisection is Better Here

- Bisection ignores function magnitudes and always halves the interval.
- Even though $f(0)$ and $f(2)$ are unbalanced, the interval shrinks symmetrically.
- So the midpoint quickly approaches $x = 1$, giving faster and more reliable convergence.

For $f(x) = x^{10} - 1$, the **false position method converges very slowly** because one function value dominates the interval, keeping the approximation near one endpoint.

The **bisection method performs better** because it is unaffected by magnitude imbalance and always halves the interval, giving steady convergence to the root.